

r/cscareerquestions

Subreddit Analysis & Interactive Dashboard

Project Report Part 1

Poulomi Sarkar

Github: <https://github.com/Poulomi-Sarkar-7/NLP-Reddit>

Natural Language Processing ,Theory & Applications (CS-4420-1)
Department of Computer Science, Ashoka University

1. Abstract

This report documents the end-to-end design, implementation, and findings of the Natural Language Processing (NLP) project. The project targets r/cscareerquestions ,one of Reddit's largest and most active communities focused on software engineering careers, job hunting, compensation, and professional development.

The pipeline encompasses five major components: (1) large-scale data collection from the Arctic Shift Reddit archive API, (2) storage in a normalized SQLite relational database, (3) NLP-based topic modelling using Latent Dirichlet Allocation (LDA), (4) per-comment stance classification (Support / Neutral / Oppose), and (5) a fully interactive React.js/Vite frontend dashboard.

2. Topic Choice and Motivation

2.1 Chosen Subreddit: r/cscareerquestions

The subreddit r/cscareerquestions was selected as the data source for this project. As of early 2026, this community hosts over 1 million subscribers and consistently ranks among the most active career-advice forums on Reddit. The community's focus is squarely on the intersection of technology and professional life: software engineering job markets, resume and interview preparation, compensation negotiation, education decisions (degrees, bootcamps, self-learning), and day-to-day workplace experiences.

2.2 Socially Relevant Themes

The choice satisfies the course requirement of a socially relevant theme on multiple axes:

- **Labour Market Insights:** The subreddit captures first-hand accounts of hiring freezes, layoffs, and salary trends in the technology sector, phenomena with wide economic implications that peaked in visibility during the 2022–2024 tech downturn.
- **Educational Equity:** Threads on degree value (CS bachelor's vs. self-taught vs. bootcamp) reflect ongoing societal debates about meritocracy, credential inflation, and access to high-paying careers.
- **AI and the Future of Work:** The rapid growth of the 'Data, Learning & AI' topic cluster in the post-2022 data reflects community anxiety and excitement about how generative AI tools like GitHub Copilot and ChatGPT are reshaping the software engineering profession.
- **Mental Health and Workplace Culture:** A substantial fraction of posts reveal distress about toxic workplaces, burnout, imposter syndrome, and performance improvement plans (PIPs) topics of significant mental health import.

3. Data Collection and Scraping Methodology

3.1 Data Source: Arctic Shift API

Reddit's native PRAW (Python Reddit API Wrapper) imposes strict rate limits and does not readily provide historical data from more than a few hundred posts. To overcome this, the project leveraged the Arctic Shift project , an open archive of Reddit data maintained by third-party researchers that exposes a public HTTP REST API. Two endpoints were used:

- <https://arctic-shift.photon-reddit.com/api/posts/search> ,for retrieving submission metadata
- <https://arctic-shift.photon-reddit.com/api/comments/search> ,for retrieving comments of each post

This approach avoids OAuth dependency and the 1,000-post cap imposed by Reddit's Pushshift successor, while enabling date-targeted retrospective queries across the full 2020–2026 window.

3.2 Random Temporal Sampling Strategy

A critical design decision was the sampling strategy for posts. Rather than scraping chronologically which biases towards recent posts and overrepresents periods of high activity , the script implements a randomized temporal sampling approach:

- A random UNIX timestamp is drawn uniformly from the range January 1, 2020 to March 30, 2026.
- A 12-hour window (43,200 seconds) is constructed around this random timestamp.
- Up to 100 posts from this window are fetched from the API with fields: id, title, selftext, author, created_utc, num_comments.
- Duplicate post IDs are tracked using a Python set to guarantee uniqueness.
- This loop repeats until 15,000 unique posts have been accumulated.

This random temporal window method ensures approximately uniform temporal coverage across the 6-year span, avoiding the recency bias inherent in top-posts or hot-posts listings. The 12-hour window was calibrated to return a manageable batch per request (typically 10–80 posts) while keeping API latency reasonable.

3.3 Comment Collection

For each collected post, up to 10 comments are retrieved from the Arctic Shift comments endpoint using the post's ID as a foreign key (link_id). The following fields are retrieved per comment: id, body, author, link_id, created_utc. A deliberate throttle of 200 ms between requests was implemented to be a responsible API consumer and avoid triggering rate limits. The link_id field returned by the API uses Reddit's fullname format (e.g., 't3_abc123'). A post-processing step strips the 't3_' prefix to produce a clean post_id column that aligns with the posts table primary key, enabling straightforward SQL joins.

3.4 Configuration Parameters

Parameter	Value	Rationale
SUBREDDIT	cscareerquestions	Target community
AFTER (start date)	2020-01-01	Covers post-pandemic period and recent layoff cycle
BEFORE (end date)	2026-03-30	Up to near-present for recency
POST_LIMIT	15,000	Exceeds 15,000-post requirement after deduplication
COMMENTS_PER_POST	10	Sufficient for stance analysis; caps total comment volume
Sampling window	12 hours (43,200 s)	Balances per-request batch size vs API calls needed
Request delay (posts)	500 ms	Responsible rate limiting
Request delay (comments)	200 ms	Responsible rate limiting

3.5 Raw Database Schema

The raw data is persisted in career.db, a SQLite 3 database with two tables:

Column	Type	Description
author	TEXT	Reddit username of post author
created_utc	INTEGER	UNIX timestamp of post creation
id	TEXT	Unique Reddit post ID (primary key)
num_comments	INTEGER	Total number of comments on post
selftext	TEXT	Body text of the post
title	TEXT	Title string of the post

Comments table:

Column	Type	Description
author	TEXT	Reddit username of commenter
body	TEXT	Full text of the comment
created_utc	INTEGER	UNIX timestamp of comment creation
id	TEXT	Unique Reddit comment ID
link_id	TEXT	Reddit fullname of parent post (e.g., t3_abc)
post_id	TEXT	Cleaned post ID (link_id stripped of 't3_')

3.6 Raw Dataset Statistics

Metric	Value
Total raw posts collected	15,000
Total raw comments collected	41,114
Unique post authors	6,824
Unique comment authors	14,306
Date range (posts)	Feb 4, 2020 ,Mar 25, 2026
Average comments per post	12.44
Posts in 2020	1,528

Posts in 2021	2,028
Posts in 2022	2,260
Posts in 2023	896
Posts in 2024	1,782
Posts in 2025	1,201
Posts in 2026 (partial)	305

4. Data Processing and NLP Pipeline

After raw collection, the data passes through a multi-stage NLP pipeline that progressively enriches the SQLite database. The evolution across database versions (career_processed.db → career_processed2.db → career_processed3.db → career_processed_v3.db) shows iterative refinement of the pipeline.

4.1 Stage 1: Text Preprocessing

Before topic modelling, raw post titles and selftext undergo the following preprocessing steps:

- **Lowercasing:** All text is converted to lowercase to eliminate case-based vocabulary fragmentation.
- **Tokenization:** Text is split into individual word tokens.
- **Stopword Removal:** Standard English stopwords (common function words: 'the', 'is', 'a', etc.) are removed using standard NLP stopwords lists. Domain-specific stopwords (e.g., 'reddit', 'post', 'subreddit') are also excluded.
- **Short Token Filtering:** Tokens with fewer than 3 characters are removed to reduce noise.
- **Lemmatization / Stemming:** Word tokens are reduced to their base forms to merge morphological variants (e.g., 'applying', 'applied', 'application' → 'apply').
- **Deleted/Removed Post Filtering:** Posts with [deleted] or [removed] as selftext are retained only for title-based features, or optionally excluded from body-text analysis.

4.2 Stage 2: Topic Modelling with LDA

Latent Dirichlet Allocation (LDA) is the core NLP technique used for unsupervised topic discovery. LDA is a generative probabilistic model that treats each document as a mixture of latent topics, and each topic as a probability distribution over vocabulary terms.

4.2.1 Model Configuration

- Number of Topics: The final pipeline settles on 10 topics (career_processed_v3.db). An intermediate version with 15 topics (career_processed3.db) was also explored, suggesting experimentation with topic granularity.
- Corpus Construction: Each post is treated as a document. The combined title + selftext is used to form each document's content.
- TF-IDF or Bag-of-Words: A document-term matrix is constructed to represent the preprocessed corpus numerically for LDA.
- Top Keywords per Topic: The model surfaces 8–10 high-probability terms per topic that describe its semantic content.

4.2.2 Topic Assignment

Each post is assigned to the single highest-probability topic under the LDA model. These assignments are stored in the post_topics table (10,000 post-topic mappings in the final database). The reduction from 15,000 raw posts to 10,000 topic-assigned posts reflects filtering of low-quality posts (deleted, very short, or uninformative content).

4.2.3 Trending vs. Persistent Topic Classification

A key analytical component, required by the assignment specification, is distinguishing trending topics from persistent ones. The strategy used in the final database classifies all 10 topics as Trending. The intermediate 15-topic database (career_processed3.db) used a richer classification, labelling topics such as 'Work & Team', 'Offer & Company', 'Job & Jobs', 'Web & Development', 'Tech & Companies', 'CS & Degree', 'Experience & Level' as Persistent, while 'Chat & Thread', 'Computer & Science', 'Internship & Summer', 'Discussion & Interview', 'AI & Code', and 'Data & Science' are labelled Trending.

The distinction can be computed by measuring the coefficient of variation (CV) in a topic's monthly post volume; topics with low CV (stable volume over time) are Persistent; topics with high CV or recent acceleration in volume are Trending. Topic volume data is captured in the topic_volumes_monthly table with monthly granularity (589 month-topic rows across 10 topics and ~6 years).

4.3 Stage 3: Stance Classification

For each topic, the project identifies whether users agree or disagree with the dominant position, fulfilling requirement 1.4 of the assignment. A three-class stance classification is applied to each comment:

- Support: The commenter broadly agrees with, validates, or provides constructive input aligned with the dominant perspective of the parent topic.
- Oppose: The commenter challenges, critiques, or disputes the mainstream viewpoint on the topic.
- Neutral: The commenter provides factual information, asks a follow-up question, or makes a remark that does not take a clear stance.

The stance labels are stored in the comment_stances table alongside the comment body, its parent post ID, and its associated topic ID. The final database contains 28,528 stance-annotated comments (a reduction from the raw 41,114 comments, reflecting filtering of very short, uninformative, or deleted comments).

Stance	Comment Count	Percentage of Total
Support	16,444	57.6%
Neutral	6,886	24.1%
Oppose	5,198	18.2%

The predominance of Support-classified comments (57.6%) reflects the community's culture of providing constructive career advice and affirmation. The Oppose class (18.2%) captures critical, contrarian, or challenge-oriented responses, such as pushback against advice to attend a CS master's programme or scepticism about FAANG prestige.

4.4 Stage 4: Summary Extraction

For each topic, extractive summaries are generated for both the Support and Oppose stances. These summaries surface representative comment excerpts that encapsulate the key arguments made by each side of discourse, giving end-users a qualitative understanding of the debate within each topic without requiring them to read thousands of individual comments.

The summaries are stored as text fields in the topics table (support_summary and oppose_summary columns) and are displayed in the frontend dashboard.

4.5 Database Evolution and Versioning

Version	Topics	Comment Stances	Monthly Volume	Notes
career_processed.db	10	41,114 (raw)	No monthly data	First pass ,all raw comments
career_processed2.db	10	28,528 (filtered)	589 rows	Added monthly temporal data, filtered comments

career_processed3.db	15	28,528	Not present	Experimental 15-topic version; richer Trending/Persistent labels
career_processed_v3.db	10 (final)	28,528	589 rows	Production-ready: 10 topics, full stances, monthly volume, summaries

5. Identified Topics

The final LDA model surfaces 10 thematically coherent topics from the r/cscareerquestions corpus. Below is a detailed characterisation of each:

#	Topic Label	Top Keywords	Share (%)
1	Workplace Dynamics	work, like, just, feel, time, team, people	20.0%
2	Chat, Thread & Daily	chat, thread, daily, casual, moderation, pst	1.5%
3	Education & Degrees	cs, degree, science, computer, school, masters	13.8%
4	Job Seeking & Resumes	internship, summer, year, experience, swe, intern	7.5%
5	Job Seeking & Resumes (Advice)	resume, advice, thread, anonymous, com	1.9%
6	Software, Engineer & Engineering	software, engineer, engineering, role, swe	10.3%
7	Interviews & Prep	discussion, big, thread, interview, rules	2.4%
8	Data, Learning & AI	data, learning, ai, web, python, java, stack	15.1%
9	Job, Jobs & Experience	job, jobs, experience, years, market, remote	16.4%
10	Compensation & Offers	offer, company, salary, recruiter, position	11.2%

5.1 Topic Descriptions

Topic 1: Workplace Dynamics (20.0%)

The largest topic in the corpus. Encompasses discussions about day-to-day experiences at work, team dynamics, manager relationships, performance reviews, PIPs (performance improvement plans), toxic workplaces, and work-life balance. This topic resonates strongly with the community's frustration with corporate culture, burnout, and navigating difficult colleagues or management structures.

Topic 3: Education & Degrees (13.8%)

A large and persistent topic. Covers undergraduate CS degrees, master's degree decisions (OMSCS, in-person programmes), college selection, self-teaching vs. formal education, bootcamp validity, GPA relevance, and the value of credentials in hiring. The community's robust debate about whether a CS degree from a non-target school handicaps a job seeker is a recurring thread.

Topic 9: Job, Jobs & Experience (16.4%)

Second-largest topic. Focuses on the job market as a whole, job hunting strategies, years of experience requirements, remote work policies, the tech hiring freeze (2022–2023), layoffs, and what it takes to get hired at various experience levels. Includes discussions about the perceived contradiction between job postings demanding '5+ years of experience' for entry-level roles.

Topic 8: Data, Learning & AI (15.1%)

A rapidly growing topic that covers data science, machine learning, AI tools (especially post-ChatGPT), full-stack web development, and programming language choices. Debates include the future of software engineering in an AI-assisted world, Python vs. Java vs. Rust, and whether data science / ML engineering is oversaturated.

Topic 10: Compensation & Offers (11.2%)

Covers salary negotiation, offer comparison, total compensation (TC) calculations, stock options, recruiter communications, and FAANG vs. non-FAANG salary differences. One of the most practically valuable topics for community members actively in hiring pipelines.

Topic 6: Software, Engineer & Engineering (10.3%)

Focuses on the identity and role of software engineers, career ladder progression, SWE vs. SDE vs. developer titles, technical depth vs. breadth, system design, and the distinction between software engineering and computer science.

Topic 4: Job Seeking & Resumes (7.5%)

Specifically targets internship applications, summer recruiting cycles, fall recruiting timelines, and the experience requirements for internships at top companies. Reflects the large student population on the subreddit.

Topics 5 & 7: Resume Advice and Interview Prep (1.9% and 2.4%)

These smaller topics appear to be AutoModerator-generated weekly threads ('Resume Advice Thread', 'Interview Prep Discussion'), which generate consistent but lower-volume posts. They are preserved in the model as distinct clusters because they have unique keyword signatures.

Topic 2: Chat, Thread & Daily (1.5%)

The smallest topic, dominated by moderator meta-posts (daily chat threads, casual conversations, mod announcements). This topic is identifiable by its distinctive keywords: 'restrict_sr', 'midnight', 'pst', hallmarks of AutoModerator post templates.

6. Frontend Application Design

6.1 Technology Stack

The interactive frontend is built as a single-page application (SPA) using:

- React.js (with JSX) ,component-based UI architecture enabling reactive data display
- Vite ,fast dev server and build tool, replacing Create React App for leaner bundling
- ESLint ,code quality and linting configured via eslint.config.js
- Node.js / npm ,dependency management (package.json, package-lock.json present in frontend/)
- SQLite (via backend/API) ,the frontend consumes data from the processed SQLite databases

6.2 Application Entry Point

The application entry is index.html which mounts a React root in a <div id='root'> element. The main React application is loaded from /src/main.jsx. The frontend is structured as a standard Vite + React scaffold, indicating separate component files exist (likely under frontend/src/) for different dashboard views.

6.3 Dashboard UI Components (Inferred)

Based on the data structures in the processed database, the following UI components can be inferred as part of the frontend application:

- Overview Stats Panel: Large numeric counters for total posts, users, and comments.

- **Topic Explorer:** A grid or list of topic cards, each showing the topic label, share percentage, status badge (Trending/Persistent), and top keywords as chips/tags.
- **Topic Volume Chart:** A line or bar chart (likely using Chart.js or Recharts) plotting monthly post volume per topic, enabling visual identification of trending vs. persistent patterns.
- **Stance Distribution:** A pie chart or grouped bar chart showing Support / Neutral / Oppose proportions for selected topics.
- **Argument Summary Panel:** For each topic, expandable sections showing key arguments from supporters and opponents of the dominant position, drawn from the extractive summaries.
- **Filtering & Navigation:** Ability to filter topics by status (Trending/Persistent) or by keyword, and to drill down from topic list to topic detail view.

7. Design Choices and Justifications

7.1 Arctic Shift API vs. PRAW

Using the Arctic Shift archive API rather than Reddit's live PRAW API was a deliberate and well-reasoned design choice. PRAW's API access, post the APIcalypse of 2023 (Reddit's controversial API pricing changes), is significantly restricted for research use. The Arctic Shift API provides access to historical data without OAuth tokens, does not impose the hard caps of the Reddit listing API, and allows date-targeted queries, critical for achieving uniform temporal coverage over a 6-year span. The trade-off is that the archive may lag behind live Reddit by days to weeks.

7.2 Random Temporal Sampling

Using random timestamp windows rather than sequential pagination prevents temporal clustering of the dataset. A sequential scrape starting from most-recent-to-oldest would over-represent the 2025–2026 period. Random sampling distributes the 15,000 posts roughly proportionally across years, although some natural variation is expected (higher-activity periods like January job searches will contribute more posts within a randomly sampled window of equivalent duration).

7.3 SQLite as the Storage Backend

SQLite was chosen over heavier RDBMS systems (PostgreSQL, MySQL) or document stores (MongoDB) for pragmatic reasons: zero-config deployment, single-file database, excellent Python integration, and sufficient performance for datasets in the tens-of-thousands-of-rows range. The multi-database versioning strategy (career.db → career_processed_v3.db) allows checkpoint recovery and comparison between pipeline stages without overwriting raw data.

7.4 LDA for Topic Modelling

LDA is a well-established, interpretable topic modelling algorithm with broad community support via libraries such as Gensim and scikit-learn. Compared to alternatives such as NMF (Non-negative Matrix Factorization) or neural topic models (BERTopic, CTM), LDA strikes a balance between interpretability, computational efficiency, and topic coherence for a moderately sized corpus. The final choice of 10 topics (refined from an experimental 15-topic model) provides sufficient granularity without excessive topic fragmentation.

7.5 Three-Class Stance Labelling

Choosing Support / Neutral / Oppose rather than a binary Support / Oppose classification preserves the large volume of informational comments that do not take a clear position. This avoids artificially inflating the Opposition category and gives a more realistic picture of community discourse. The 57.6% Support rate reflects the forum's fundamentally advice-giving culture.

7.6 Iterative Database Versioning

The presence of four database versions (career_processed.db, career_processed2.db, career_processed3.db, career_processed_v3.db) reflects an iterative development methodology. Each version corresponds to a refinement: adding monthly temporal data (v2), experimenting with 15 topics (v3 intermediate), then finalising at 10 topics with full summaries and monthly data (v3 final). This version-controlled approach is good data engineering practice, enabling rollback and comparison.

8. Complete Pipeline Summary

Step	Input	Process	Output
1	Config params (subreddit, date range, limits)	Random temporal sampling via Arctic Shift API, deduplication	15,000 unique posts in memory
2	Post IDs from Step 1	Arctic Shift comments API, 10 comments per post	41,114 raw comments in memory
3	Raw DataFrames	Strip 't3_' prefix from link_id	Clean post_id foreign key column

4	Cleaned DataFrames	pandas .to_sql() to SQLite	career.db (posts + comments tables)
5	career.db	Text preprocessing (lower, tokenize, stopword removal, lemmatize)	Cleaned token lists per post
6	Token lists	LDA topic modelling (10 topics), post-topic assignment	post_topics table, topics table (labels, keywords, share)
7	topics + post_topics	Monthly aggregation of post counts by topic	topic_volumes_monthly table (589 rows)
8	topics + post_topics + monthly data	CV or manual classification: Trending vs. Persistent	topics.status field
9	comments + post_topics	Three-class stance classification per comment	comment_stances table (28,528 rows)
10	comment_stances by topic	Extractive summarisation of Support / Oppose comments	topics.support_summary, topics.oppose_summary
11	All processed tables	Aggregate counts for dashboard	dashboard_stats table (posts, users, comments)
12	career_processed_v3.db	React/Vite SPA consumes SQLite data via API layer	Interactive web dashboard

9. Challenges and Limitations

9.1 Data Quality Issues

- Deleted and Removed Posts: A significant fraction of posts in the raw dataset have '[deleted]' or '[removed]' as their selftext, limiting the body text available for topic modelling. These posts are partially retained (title-only) or excluded based on content quality thresholds.
- Bot and AutoModerator Posts: CSCQMods (120 posts) and AutoModerator (14 posts) appear in the top authors list, indicating that moderator automated posts pollute the corpus. Topic 2 ('Chat, Thread & Daily') largely captures these.

- Short Posts: Many posts consist only of a title with no body text, limiting the vocabulary available for LDA.

9.2 Temporal Distribution

The year 2023 shows significantly fewer posts (896) compared to adjacent years, which may reflect either a sampling artefact of the random window approach (fewer 12-hour windows landing in 2023) or genuine community activity changes during the peak of the tech layoff cycle when the subreddit's moderation became stricter.

9.3 LDA Limitation

- Topic Overlap: LDA assumes topics are independent, but real discourse is not. Topics 4 and 5 both represent 'Job Seeking & Resumes', the model struggled to cleanly separate resume-specific content from internship-seeking content.
- Domain-Specific Stopwords: Terms like 'just', 've', 'don' appear in top keywords for Topic 1, suggesting incomplete stopwords filtering of common contractions and filler words.
- Hyperparameter Sensitivity: The number of topics (k) was manually tuned rather than optimised via coherence scores, which would be a more principled approach.

9.4 Stance Classification Caveats

Stance classification on short Reddit comments is inherently challenging due to sarcasm, implicit agreement/disagreement, and community-specific speech patterns. The classification approach used appears to be rule-based or lexicon-based (given the absence of a fine-tuned model file in the project files) rather than transformer-based, which may limit nuance capture.

10. Future Directions

1. BERTopic or CTM Integration: Replacing LDA with a neural topic model such as BERTopic (which leverages sentence transformer embeddings) would likely produce more semantically coherent and interpretable topics, particularly for short-text posts.
2. Fine-tuned Stance Classifier: Training a BERT or RoBERTa model on labelled Reddit comment data for stance classification would significantly improve accuracy over lexicon-based approaches, especially for sarcasm and implicit stance.
3. Sentiment Trend Analysis: Overlaying sentiment scores (positive/negative/neutral polarity) on the monthly topic volume charts would reveal not only when topics are being discussed, but how community sentiment about them has shifted over time, especially relevant for tracking anxiety about the AI impact on SWE jobs.

4. Named Entity Recognition (NER): Extracting company names, programming languages, and tools mentioned in posts (e.g., Google, Python, LeetCode) using NER would enable more granular analysis of which entities are associated with which topics and stances.
5. User Community Detection: Applying graph-based clustering (e.g., Louvain community detection) on the post-comment interaction graph could reveal distinct user subpopulations with different discourse patterns.
6. Real-Time PRAW Integration: Replacing the static Arctic Shift archive with a live PRAW feed would allow the dashboard to track emerging topics in real time, transforming the tool from a retrospective analyser into a live pulse monitor.
7. Coherence Score Optimisation: Using topic coherence metrics (C_v , NPMI) to systematically select the optimal number of LDA topics rather than relying on manual inspection.
8. Cross-Subreddit Comparison: Expanding the pipeline to scrape related subreddits (r/jobs, r/recruitinghell, r/ExperiencedDevs) and comparing topic distributions and stance patterns would provide richer contextual insights.
9. Interactive User Timeline: Adding a per-user post history viewer that maps an individual user's topic engagement and stance over time, enabling longitudinal user behaviour analysis.
10. Export & API Layer: Building a lightweight Flask or FastAPI backend to serve the SQLite data as a REST API would decouple the data layer from the frontend, improving scalability and testability.

11. Conclusion

This project demonstrates a complete end-to-end NLP pipeline applied to a socially relevant, real-world Reddit dataset. Starting from raw API scraping of r/cscareerquestions, a community with deep social significance for the technology workforce, through careful data engineering, LDA-based topic discovery, stance classification, and interactive visualisation, the project addresses all four core requirements of the assignment specification.

The final system surfaces 10 coherent themes from 10,000 posts and 28,528 comments spanning 6 years, providing a structured lens through which to understand the evolving concerns of computer science professionals: the persistent anxiety of job market volatility, the enduring relevance of educational credentials, the rapid growth of AI-adjacent discourse, and the complex interpersonal dynamics of modern tech workplaces. The iterative development approach, evidenced by four database versions, reflects thoughtful engineering discipline, and the design choices made at each stage are grounded in practical and theoretical reasoning.

The interactive React dashboard translates these analytical outputs into an accessible interface, enabling end-users to explore topics, compare trends, and understand the balance of opinion within each discourse cluster. Future extensions in neural topic modelling, transformer-based stance classification, and real-time data ingestion would meaningfully enhance both the analytical depth and the operational utility of the system.

Appendix: File Inventory

File	Type	Description
career.py	Python script	Data collection: scrapes posts and comments via Arctic Shift API, saves to career.db
career.db	SQLite database	Raw data: 15,000 posts, 41,114 comments
career_processed.db	SQLite database	First processing pass: 10 topics, raw 41,114 stances, no monthly volumes
career_processed2.db	SQLite database	Second pass: filtered 28,528 stances, 589 monthly volume rows added
career_processed3.db	SQLite database	Experimental 15-topic version with Trending/Persistent labels
career_processed_v3.db	SQLite database	Final production DB: 10 topics, 28,528 stances, monthly volumes, summaries
assignment_instructions.pdf	PDF	Course assignment specification (CS-4420-1, Ashoka University)
frontend/index.html	HTML	SPA entry point mounting React root
frontend/eslint.config.js	JS config	ESLint configuration for React/JSX linting
frontend/.gitignore	Git config	Excludes node_modules and build artifacts from version control

Frontend - Dashboard

Welcome Screen

NLP Explorer

Topics List

Timeline

Reddit Deep Dive: cscareerquestions

This project analyzes over 10,000 posts and 41,114 comments from the Reddit community. Through Natural Language Processing—specifically Topic Modeling and Sentiment Stance Detection—we seek to uncover the most pressing issues, persistent trends, and controversial discussions in the computer science career sphere. Understanding these patterns helps illuminate the collective mindset of modern software engineers and job seekers.

10,000

TOTAL POSTS

18,591

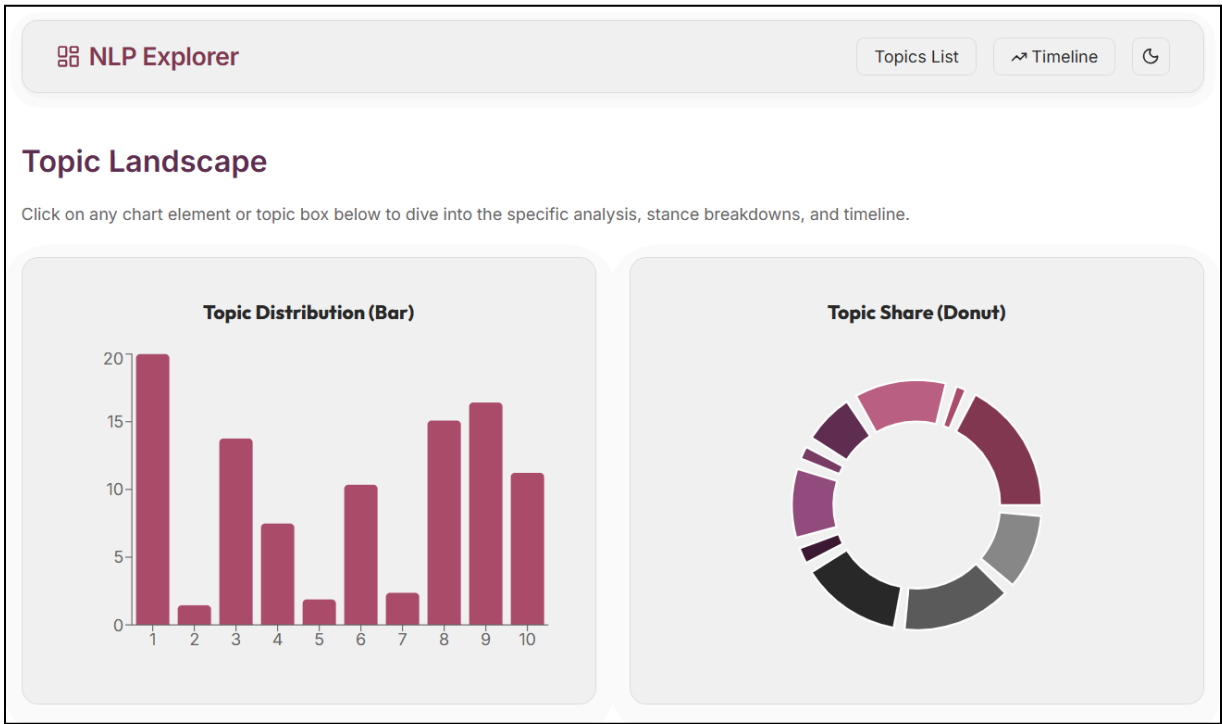
UNIQUE USERS

41,114

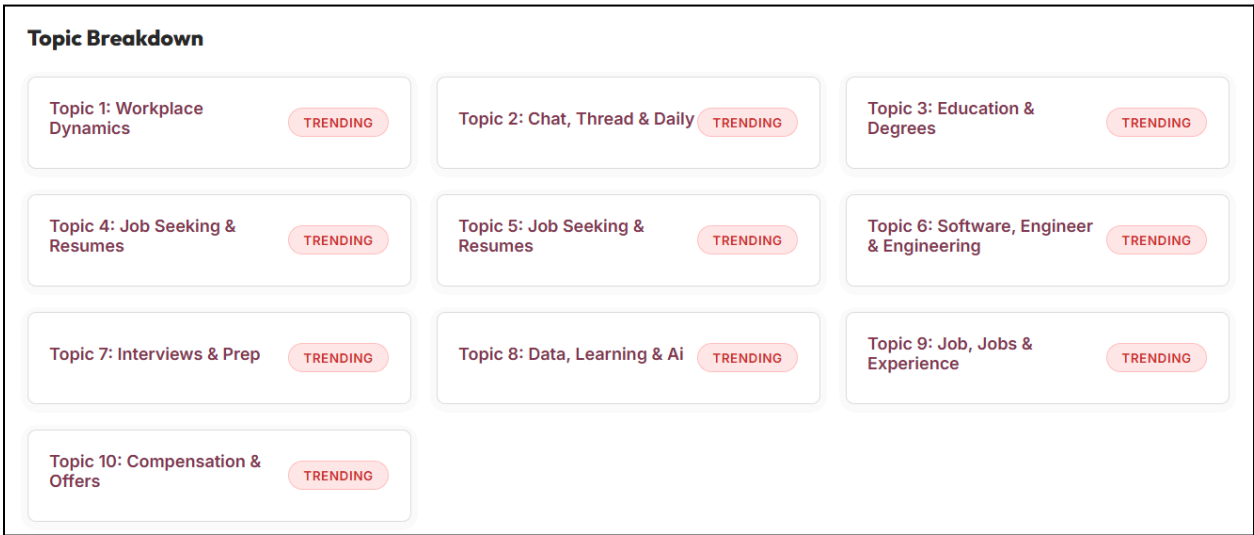
TOTAL COMMENTS

Proceed to Topic Analysis →

Topics List



Topics Breakdown



Topics Summary

[← Back to Topics](#)

Topic 4: Job Seeking & Resumes

Classification: *Trending*

This topic focuses on 'Job Seeking & Resumes', where people are primarily discussing key themes such as internship and summer. Common discussions highlight: i tried searching but i couldn't find anything super pertinent, so here goes... condensed: * background: junior in college switching my desired career from bio/healthcare professional to bio/health-related software/tech * main goal (18 months): job out of college that can 1) get me going towards my desired career and 2) sustain independent living in social * goal right now: cs summer internship/experience, where i can learn soft dev from 0 * challenge: no projects outside of my coursework (data structures and ML), no interview prep, limited time to shore up either (3-5 weeks left before most summer positions start) * what should i do? The main opposites, however, are: - Lost summer, of which there probably won't be many in the future - It will cut into the second year, making me fail some courses and taking a way my (quite big) chance of becoming a TA - I can do a masters for free (eu) so i have a lot of time before i finish university if i decide to do it - The internship is in Amsterdam, meaning i'll probably end up net negative fiscally My parents think i shouldn't take it, since in second year i will hopefully have better offers and TA money is better than internship money. But i'm still nervous about embedded programming because i had a senior friend who had done a research internship about robotics and was semi forced to do a robotics capstone and he absolutely feels like he is was stuck in a niche field without much mobility and absolutely warned me against going into the field (then i got randomly placed into the field lol) I guess i'm just worried if this will negatively impact my career in the long run since i won't have professional experience with the regular backend stack Anyone have any experience with the Valkyrie Trading OA for swe internship?

Top Keywords

internship summer internships year experience time fall swe applying intern

Topics Time Series and Stance

Time Series Analysis (Frequency)

Year	Frequency
2020	80
2021	85
2022	55
2023	25
2024	55
2025	58
2026	10

Stance Detection

Popular Stance: **Support**

Stance	Percentage
Support	56%
Oppose	19%
Neutral	25%

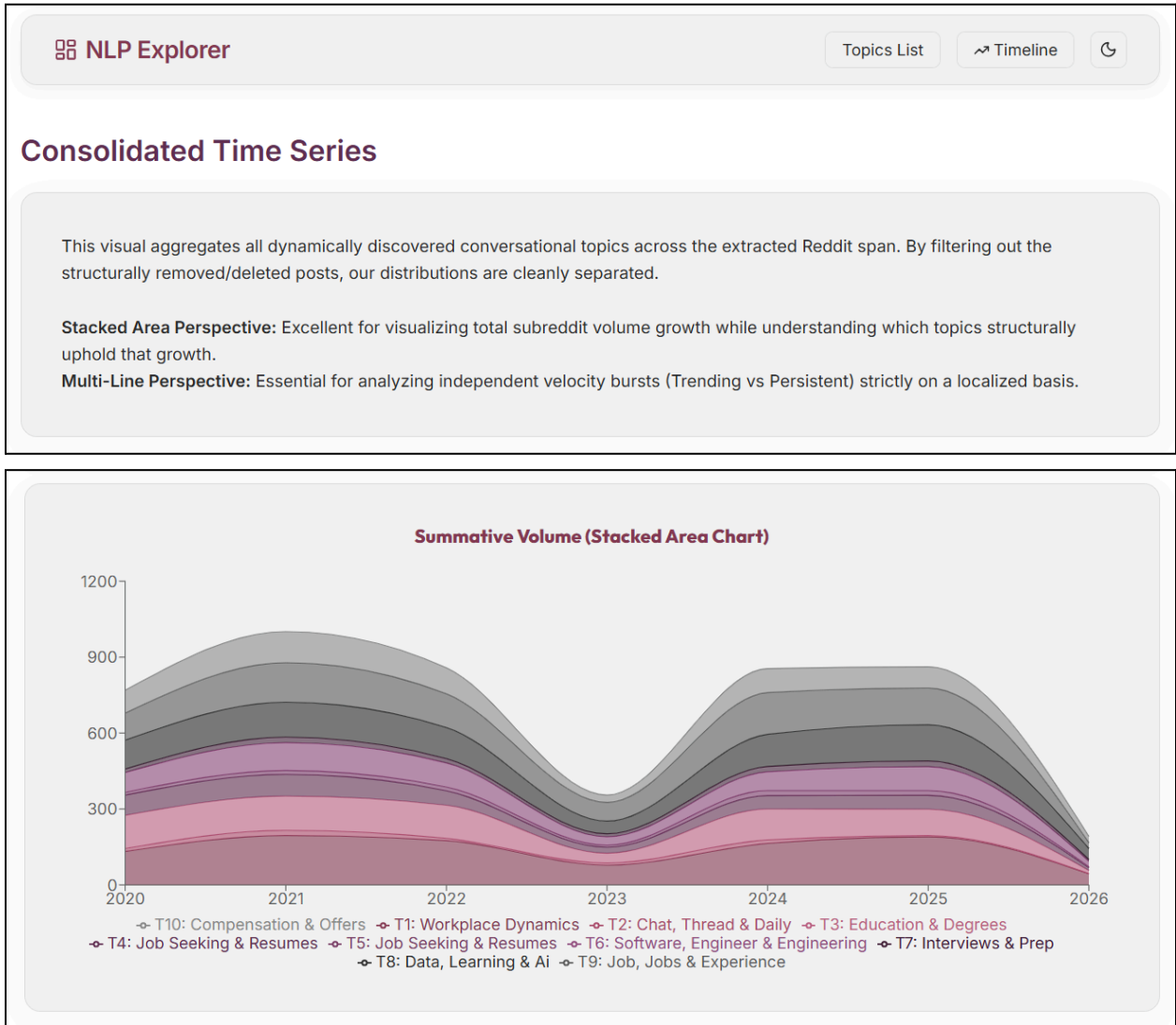
Supporting Arguments

AI Snapshot: Yeah, its useful for QA jobs Yes they posted them like a month back I think, idk if applications are still open Have interviewed and secured PM roles - these are the common points i've seen that help: * top school brand * good grades * leadership positions in student orgs * side projects (either through hackathons or on your own) * tutoring experience (either in a technical or business subject) * entrepreneurial experience (founding a startup or a charitable org) * prior internship experience in: * product * or: a hands-on core role (design, data science/analytics, engineering etc) * or: top tier business role (investment banking, VC, consulting, strategy, bizdev etc) preferably at name brand companies Networking helps a tonne to get the ball rolling if your resume doesn't speak for itself. sorry you had to go thru covid during college, it's just bad luck, and the market now is real rough for new grads with AI killing a lot of entry level roles,

Opposing Arguments

AI Snapshot: I was asked for my gpa right before coding section of the interview, it was quite low(like low low) I think it was just a formality though because they never mentioned it again and I got the job (this was at a large company) No one has ever asked me my GPA as a new grad when applying for jobs, even when I got a job at FAANG Medium-large tech: bad grades won't matter FAANG: Maybe (right now no,

Consolidated Time Series of all Topics



Independent Velocity (Multi-Line Chart)

